

Title: Order Management System

Objective / Aim

To write a Java program demonstrating classes, constructors, inheritance, method implementation, and the creation and management of special and normal orders.

System Requirements

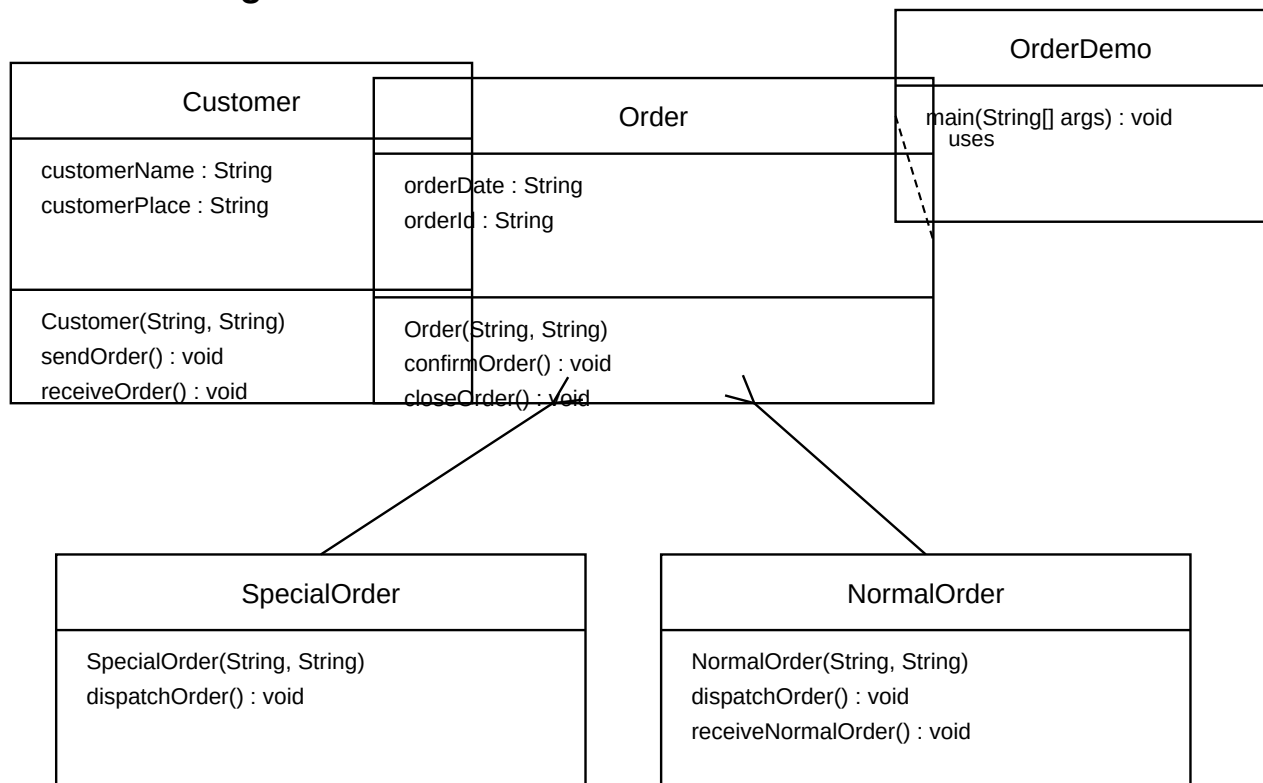
Hardware: Standard PC setup with Core i3/i5 processor and 4GB/8GB RAM.

Software: Windows/Linux/macOS, JDK 17/JDK 21, and Eclipse IDE or any Java IDE.

Algorithm / Logic Description

1. Create a Customer class with customer name and customer place.
2. Define sendOrder() and receiveOrder() methods in the Customer class.
3. Create an Order class with order date and order ID.
4. Define confirmOrder() and closeOrder() methods in the Order class.
5. Create SpecialOrder as a subclass of Order and define dispatchOrder().
6. Create NormalOrder as a subclass of Order and define dispatchOrder() and receiveNormalOrder().
7. Create Customer, SpecialOrder, and NormalOrder objects in the main() method.
8. Display customer information and perform the required special and normal order activities.

UML Class Diagram



Source Code

```

class Customer {
    String customerName;
    String customerPlace;
    Customer(String customerName,String customerPlace) {
        this.customerName=customerName;
        this.customerPlace=customerPlace;
    }
    void sendOrder() {
        System.out.println(customerName+" has sent the order.");
    }
    void receiveOrder() {
        System.out.println(customerName+" has received the order.");
    }
}

class Order {
    String orderDate;
    String orderId;

    Order(String orderDate,String orderId) {
        this.orderDate=orderDate;
        this.orderId=orderId;
    }

    void confirmOrder() {
        System.out.println("Order "+orderId+" confirmed.");
    }

    void closeOrder() {
        System.out.println("Order "+orderId+" closed.");
    }
}

class SpecialOrder extends Order {
    SpecialOrder(String date,String id) {
        super(date,id);
    }

    void dispatchOrder() {
        System.out.println("Special Order "+orderId+" dispatched.");
    }
}

class NormalOrder extends Order {
    NormalOrder(String date,String id) {
        super(date,id);
    }

    void dispatchOrder() {
        System.out.println("Normal Order "+orderId+" dispatched.");
    }

    void receiveNormalOrder() {
        System.out.println("Normal Order "+orderId+" received.");
    }
}

public class OrderDemo {
    public static void main(String[] args) {
        Customer c1=new Customer("Ravi Teja","Srikakulam");
        SpecialOrder s1=new SpecialOrder("11-08-2026","S101");
    }
}

```

```

NormalOrder n1=new NormalOrder("11-08-2026","N101");
System.out.println("Customer Information");
System.out.println("Customer Name: "+c1.customerName);
System.out.println("Place: "+c1.customerPlace);
System.out.println("\nSpecial Order Details");
s1.confirmOrder();
s1.dispatchOrder();
s1.closeOrder();
System.out.println("\nNormal Order Details");
n1.confirmOrder();
n1.dispatchOrder();
n1.receiveNormalOrder();
n1.closeOrder();
System.out.println("\nCustomer Activities");
c1.sendOrder();
c1.receiveOrder();
}
}

```

Compilation & Execution Commands

```

javac OrderDemo.java
java OrderDemo

```

Output Screen

```

PS C:\Users\Student> javac OrderDemo.java
PS C:\Users\Student> java OrderDemo
Customer Information
Customer Name: Ravi Teja
Place: Srikakulam

Special Order Details
Order S101 confirmed.
Special Order S101 dispatched.
Order S101 closed.

Normal Order Details
Order N101 confirmed.
Normal Order N101 dispatched.
Normal Order N101 received.
Order N101 closed.

Customer Activities
Ravi Teja has sent the order.
Ravi Teja has received the order.
PS C:\Users\Student>

```